Rúbrica de Evaluación de Proyecto de Desarrollo de Software

Categoría	Criterio de Evaluación	Excelente (5 pts)	Aceptable (3 pts)	Insuficiente (1 pt)
Código	**Nombres Reveladores**	El código responde preguntas por sí mismo; usa nombres precisos que revelan intención.	Algunos nombres son genéricos, pero la lógica es comprensible tras un análisis breve.	Usa nombres ambiguos (ej. `list1`, `x`, `4`) que generan dudas sobre su propósito.
Código	**Patrones de Diseño**	Aplica patrones GoF, arquitectónicos (MVC, Pub/Sub) y de implementación de forma justificada.	Utiliza algunos patrones, pero la estructura general es rígida o poco reutilizable.	No se evidencia el uso de patrones; el código carece de una estructura organizada.
Proceso	**Pruebas Automatizadas**	Cobertura >90%; las pruebas unitarias se ejecutan varias veces al día como red de seguridad.	Existe una suite de pruebas, pero la cobertura es parcial o no se ejecutan frecuentemente.	No hay pruebas automatizadas o son casi inexistentes ("Si no está probado, no existe").

Proceso	**Control de Versiones y Entrega**	Uso profesional de Git (GitHub/GitLab) y flujo de entrega continua automatizado.	Se usa control de versiones, pero de forma local o sin una estrategia de despliegue clara.	Código disperso en laptops, Dropbox o discos locales; sin control de versiones profesional.
Gestión	**Planificación y Equipo**	Equipo multidisciplinario (2-4 miembros); planificación por capas usando Historias de Usuario.	Roles rígidos en el equipo; planificación poco detallada a corto plazo.	Sin planificación clara; el equipo no tiene coach ni Product Owner definido.
Optimización	**Criterio de Optimización**	Solo se optimiza si un requisito no se cumple, usando un *Profiler* para hallar cuellos de botella.	Se optimiza basándose en suposiciones, no en datos estadísticos.	Se realiza optimización prematura sin haber validado primero la funcionalidad.
Integración	**API**	Utiliza GET, POST, PUT, DELETE según su idempotencia y devuelve códigos de estado HTTP precisos (200, 201, 204, 400, 404, 500).	Usa los verbos adecuados en su mayoría, pero devuelve códigos de estado genéricos (ej. retorna 200 OK incluso en errores de cliente).	Uso incorrecto de verbos (ej. GET para modificar datos) y omisión de los códigos de estado HTTP adecuados.